Ingeniería de Aplicaciones Web

Diego C. Martínez

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur

Módulos y Frameworks

Las Aplicaciones Web son sistemas complejos
y requieren estrategias modulares

JS

Principalmente
Function
Components

Cada lenguaje tendrá sus mecanismos
Cada framework tendrá
su filosofía arquitectónica

Las Aplicaciones Web son sistemas complejos
y requieren estrategias modulares

JS

Principalmente
Function
Components

JS

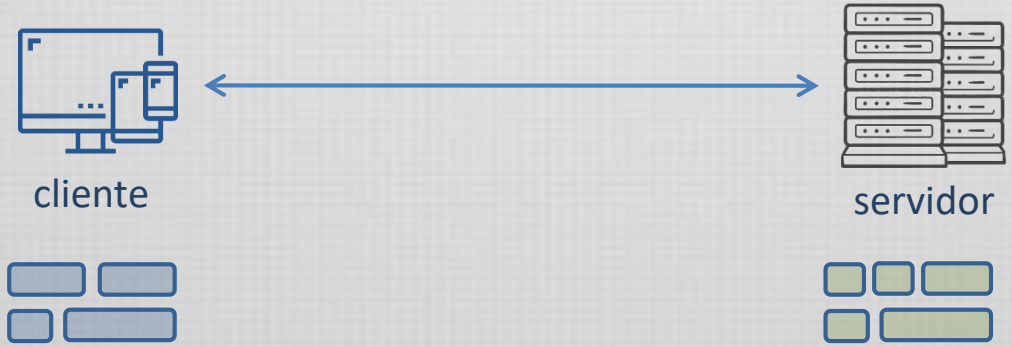
Principalmente
Function
Components

DIFERENCIA

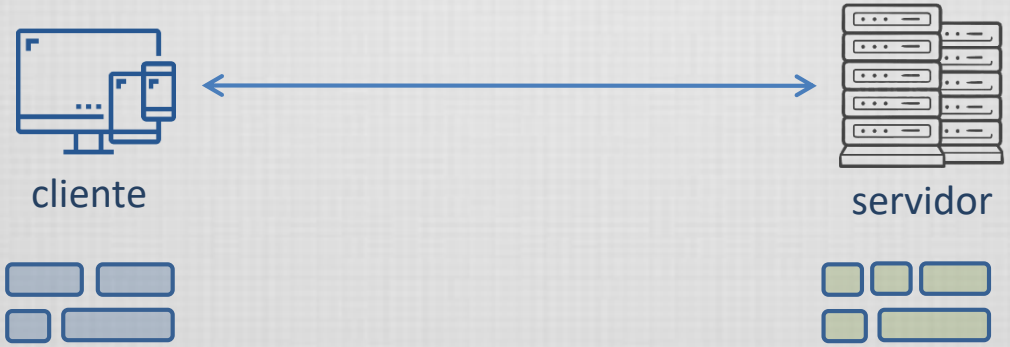
En el frontend, los componentes modelan la interfaz
En el backend, los componentes modelan lógica

Comparten la forma, pero no la responsabilidad.

Módulos y Frameworks



Componentes en Next



Client Components

Si el código necesita interactuar con el usuario (estado, clicks, efectos) va del lado del cliente

```
'use client'
```

Puede usar "cosas" del navegador
(window, document, localStorage…)

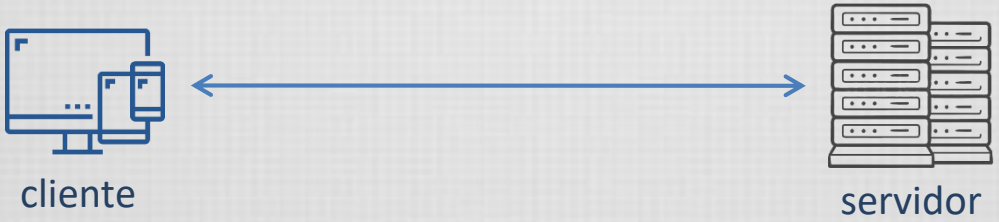
Puede usar "cosas" de UI
(onClick, onChange, etc)

Puede usar Hooks de React
(useState, useEffect,…)

Server Components

Si el código necesita acceder a la base de datos, secretos o simplemente renderizar contenido, mejor que quede en el servidor

Componentes en Next




```
'use client';
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Contador: {count}</p>
      <button
        onClick={() => setCount(count+1)}>
        Incrementar
      </button>
    </div>
  );
}
```

Next prepara la lógica necesaria para que esto se compute en el cliente

Componentes en Next

```
<div>
 <p>Contador: {count}</p>
 <form action={increment}>
   <button type="submit">
     Incrementar
   </button>
 </form>
</div>
```

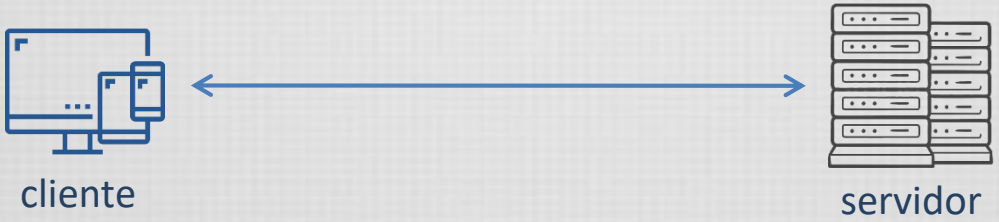
```
import { revalidatePath } from 'next/cache';
let count = 0;

async function increment() {
  'use server';
  count++;
  revalidatePath('/');
}

export default function Counter() {
  return (
    <div> <p>Contador: {count}</p>
      <form action={increment}>
        <button type="submit">Incrementar</button>
      </form>
    </div>
  );
}
```

Next prepara la lógica necesaria para que la comunicación sea via HTTP

Componentes en Next




```
<div>
 <p>Contador: 0</p>
 <form action=POST>
   <input type="hidden"
        name="$ACTION_ID_007…">
   <button type="submit">
     Incrementar
   </button>
 </form>
</div>
```

```
import { revalidatePath } from 'next/cache';
let count = 0;

async function increment() {
  'use server';
  count++;
  revalidatePath('/');
}

export default function Counter() {
  return (
    <div> <p>Contador: {count}</p>
      <form action={increment}>
        <button type="submit">Incrementar</button>
      </form>
    </div>
  );
}
```

useEffect

Permite implementar efectos "secundarios" luego del renderizado
Usualmente llamadas a APIs o actualizaciones del DOM

Acción a realizar

Bajo qué cambios

```
useEffect( función , [dependencia])
```

useContext

Permite definir datos "contextuales" que pueden ser accedidos por cualquiera que *recibe* ese contexto

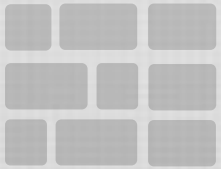
Valor contextual

```
<Context.Provider value="value">

        <ComponenteQueRecibeContexto/>

</Context.Provider
```

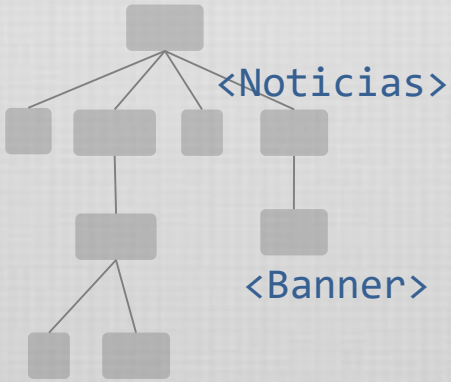
Comunicación entre componentes



Comunicación entre componentes

```
import Banner from "./Banner"
export default function Noticias() {
  return (
    <div>
      Hola!
      Usamos aquí banner
      <Banner />
    </div>
);
}
```

```
export default function Noticias() {
  return (
    <div>
      Hola! Aca cualquier tag
      {children}
    </div>
);
}
```

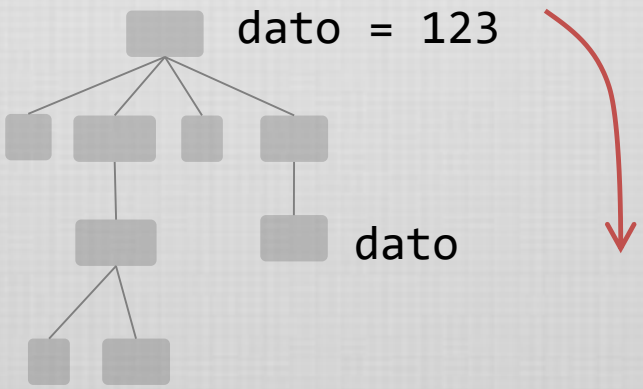



```
export default function Banner() {
  return <div>Un banner</div>;
}
```

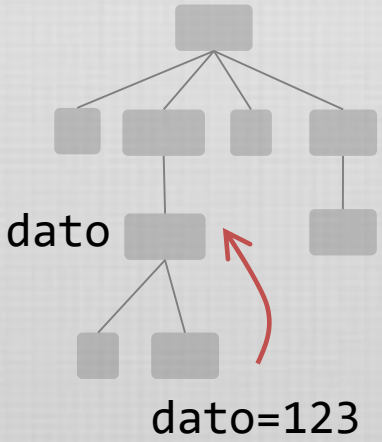
```
...
<Noticias>
...
```

```
...
<Noticias>
  <Banner>
</Noticias>
...
```

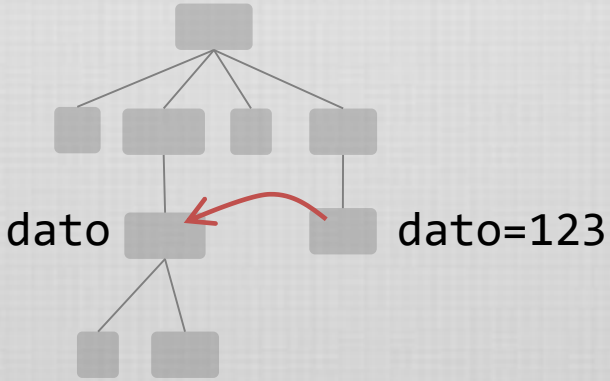
Comunicación entre componentes



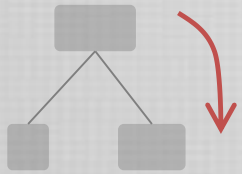
Comunicación entre componentes



Comunicación entre componentes



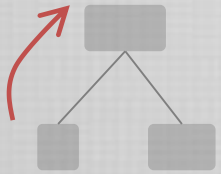
Comunicación entre componentes



```
export default function Padre() {
  return <Hijo mensaje="Yo soy tu padre" />;
}
```

```
export default function Hijo({mensaje}) {
  return <p>El mensaje del padre es: {mensaje}</p>;
}
```

Comunicación entre componentes



```
export default function Padre() {

  const manejarMensaje = (dato) => {
    alert(`Dato recibido: ${dato}`);
  };

  return <Hijo onEnviar={manejarMensaje} />;
}
```

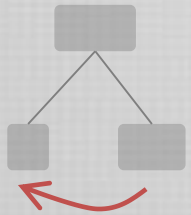
El padre define una función

... y se la pasa al hijo

```
export default function Hijo({onEnviar}) {
  return (
    <button onClick={() => onEnviar("Hola desde el hijo")}>
      Enviar mensaje al padre
    </button>
  );
}
```

El hijo usa esa función para enviar datos al padre

Comunicación entre componentes



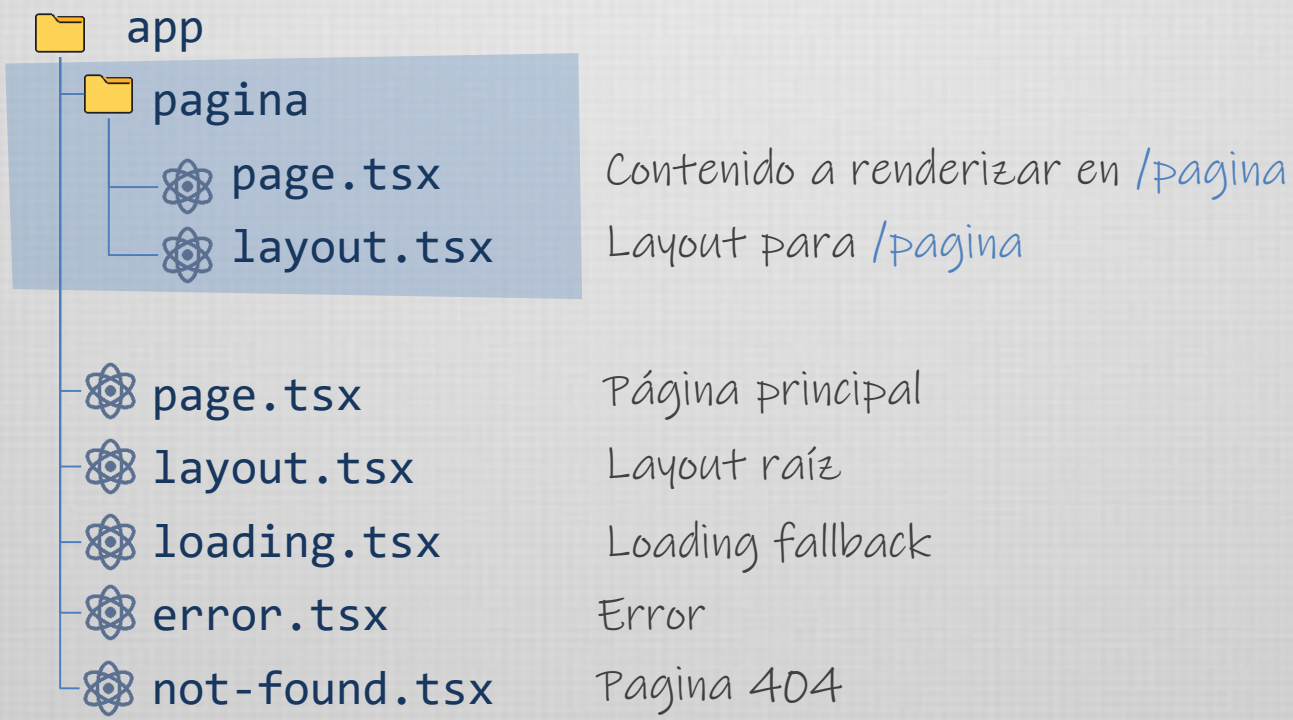
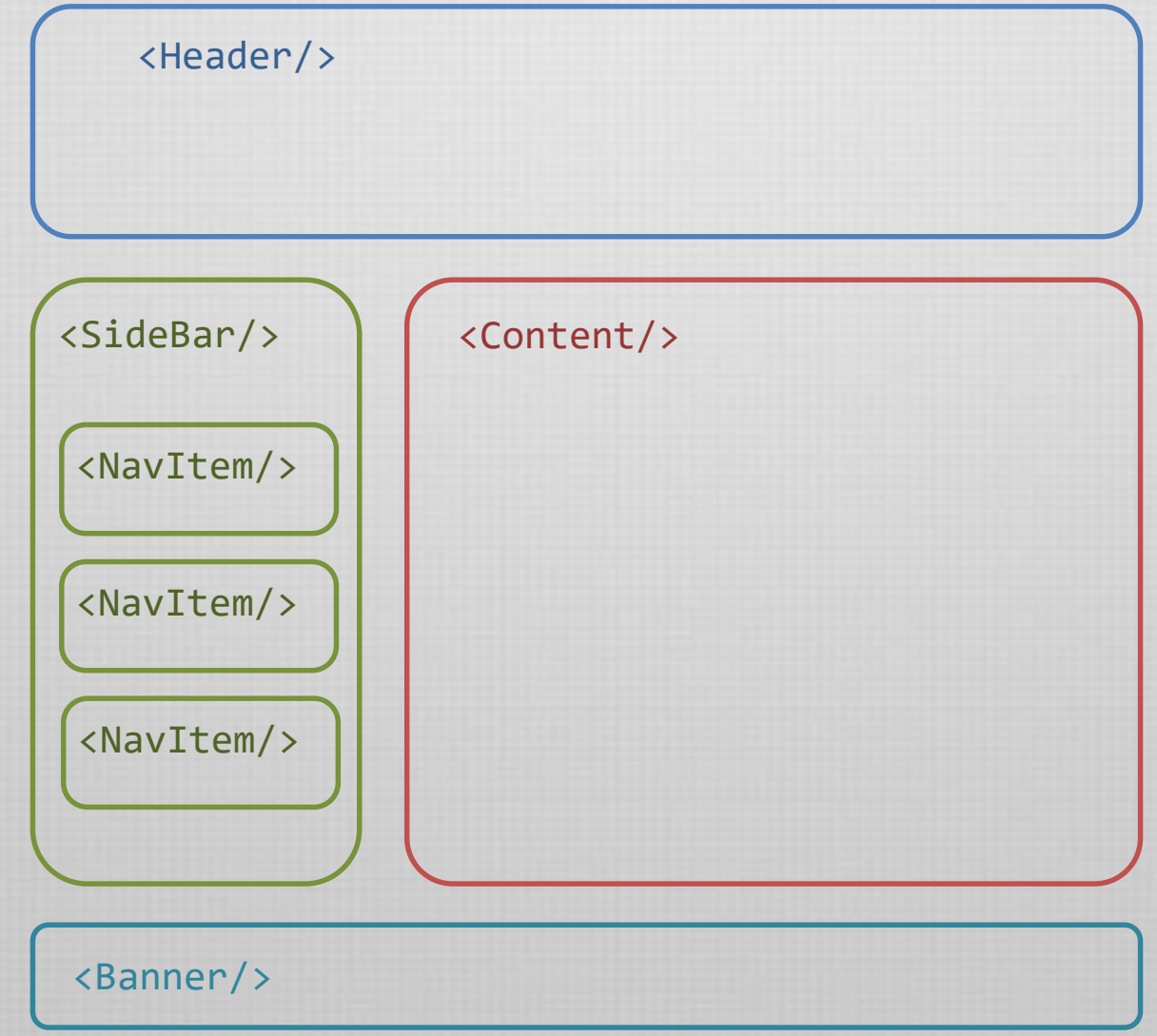
Opción #1

El padre mantiene un estado que el hijo puede modificar
y el hermano puede observar
(Una combinación de los dos anteriores)

Opción #2

Definir un contexto y que sea accesible para los dos hermanos

Estructura de carpetas de NextJS



Next